



CMPT 295

Unit – Microprocessor Design & Instruction Execution

Lecture 32 – Pipelined Execution + Hazards

Last lecture

Microprocessor machine instruction execution:

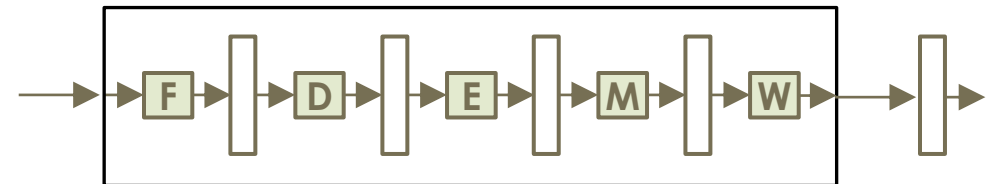
Model 1. Sequential execution of machine code instructions

- **Single-cycle** microprocessor executing one machine code instruction **at a time** and **per clock cycle**
- **Result:** long clock cycle, small throughput

How to improve throughput?

Model 2. Staged execution of machine code instructions

- Divide the execution of instructions into stages: **fetch**, **decode**, **execute**, **memory**, **write back**
- Introduce pipeline registers after each stage
- **Result:** Shorter (faster) clock cycle
-> faster computer 😊
- Issues:



- Adding **pipeline registers** increase **latency**
- Stages may not have same propagation delay

Today's Menu

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design principles
 - Look at an example of an instruction set: MIPS
 - Create our own
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine code instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine code instructions
 - Staged execution of machine code instructions
 - Pipelined execution of machine code instructions + Hazards

Let's practice a little!

1. Minimum Clock Cycle:
Latency:
Throughput:
CPI:

2. Minimum Clock Cycle:
Latency:
Throughput:
CPI:

3. Minimum Clock Cycle:
Latency:
Throughput:
CPI:

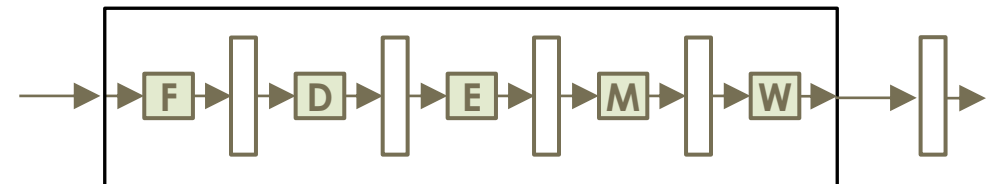
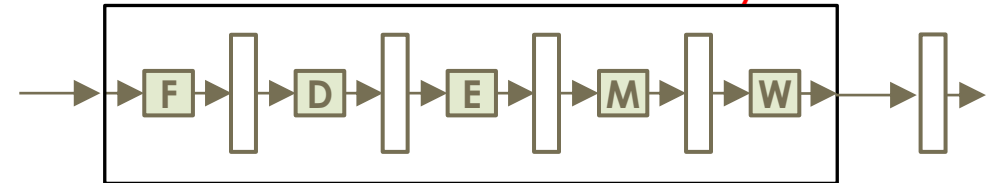
Model 1. Sequential execution of machine code instructions

=> Microprocessor executes one machine instruction **at a time** and **per clock cycle**



Model 2. Staged execution of machine code instructions

=> Microprocessor executes one machine instruction **at a time** over **a few clock cycles**



How to improve our design:

Back to the *cafeteria*

- The *cafeteria* is divided into sections
- It takes the same amount of time for one customer to go through these sections whether she is alone in the queue or not
- **So**, the *cafeteria* would be more *time efficient* (i.e., higher customer *throughput*) if there were more than one customer going through the sections of the *cafeteria*, i.e., more customers in the queue
 - More customers would be served in the same amount of time
 - How many customers would this be?
 - In **steady-state**, there would be as many customers as there are sections!
 - Therefore, the highest customer *throughput* would be achieved by having **one customer per section**

Steady-state: there is a customer in each section of the *cafeteria*.

Model 3: Pipelined execution of machine code instructions

- Example: cafeteria with 1 customer in the queue (Model 2) versus cafeteria with many customers in the queue (1 customer in each section -> “steady-state”)

Model 2.
Staged
Execution

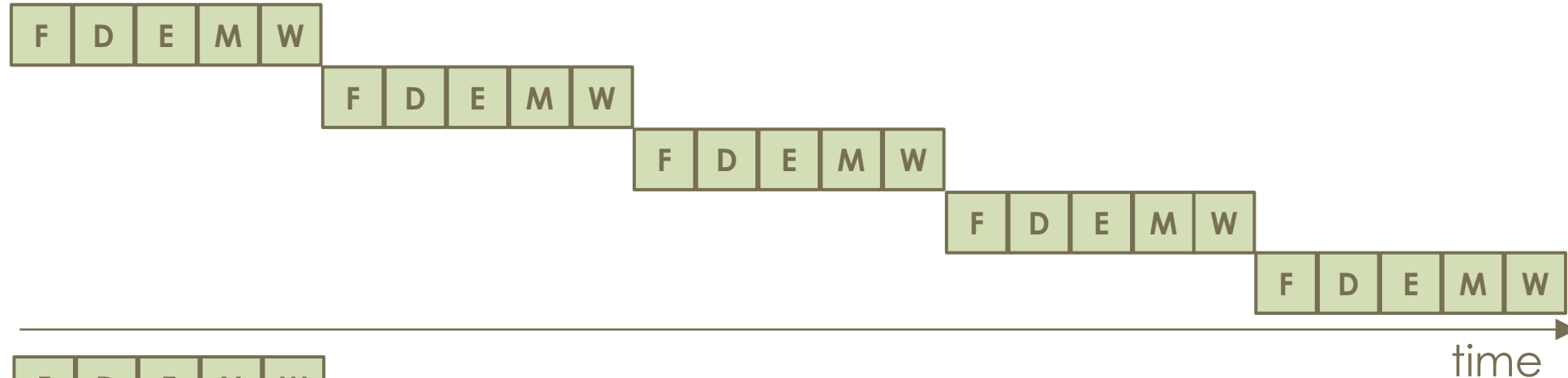
l_1 :

l_2 :

l_3 :

l_4 :

l_5 :



Model 3.
Pipelined
Execution

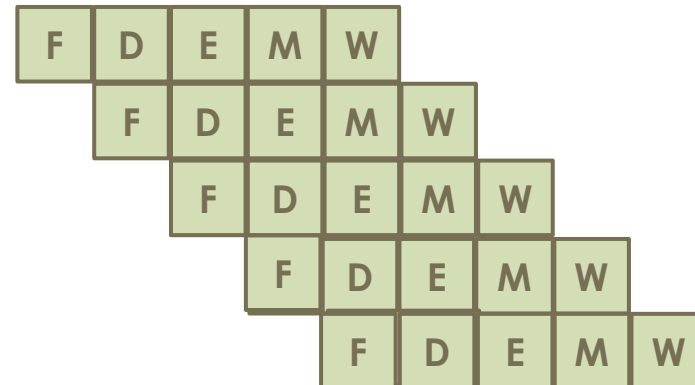
l_1 :

l_2 :

l_3 :

l_4 :

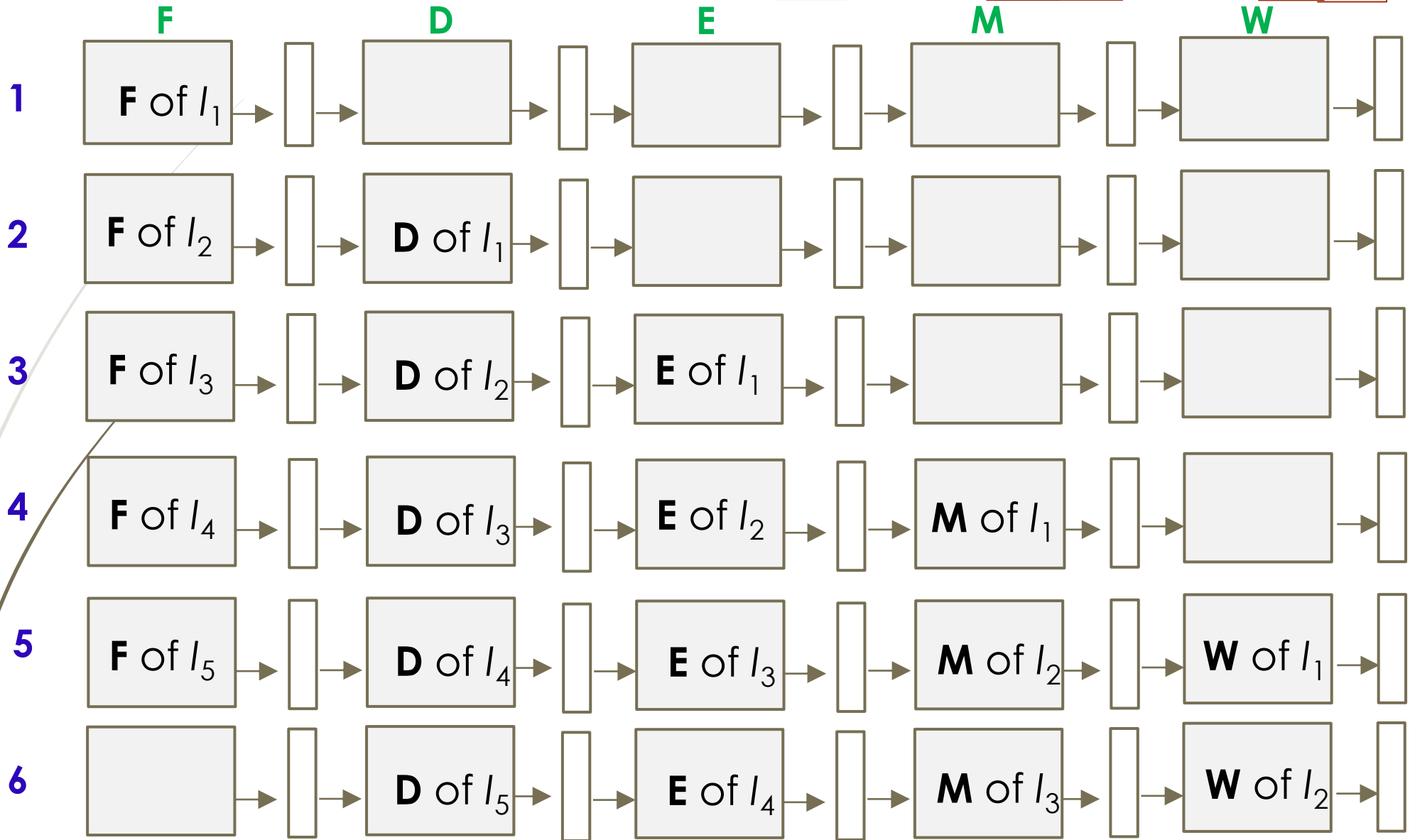
l_5 :



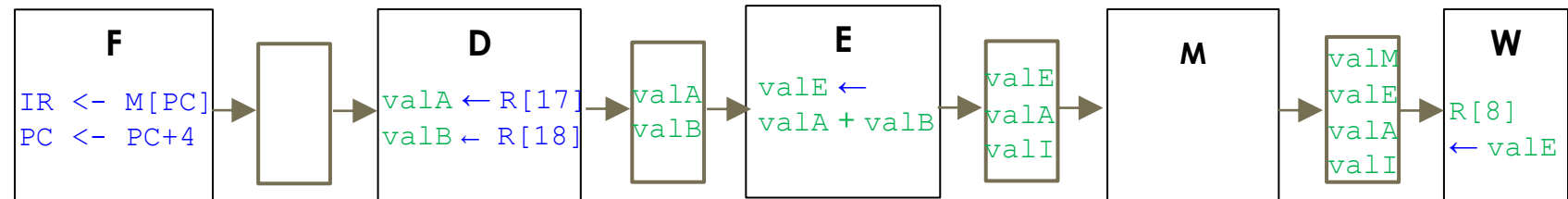
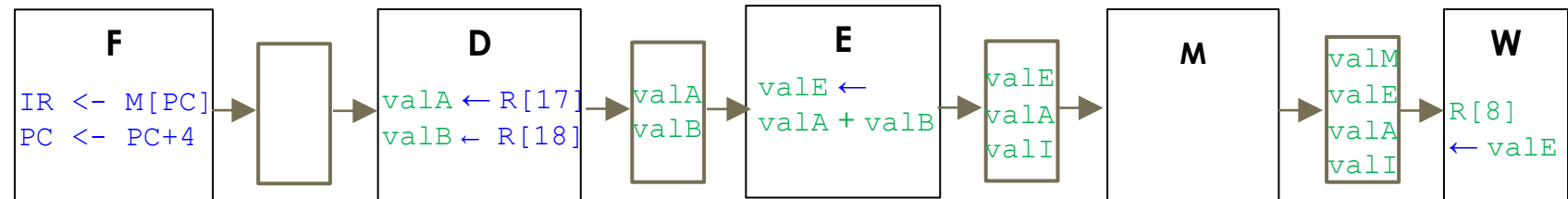
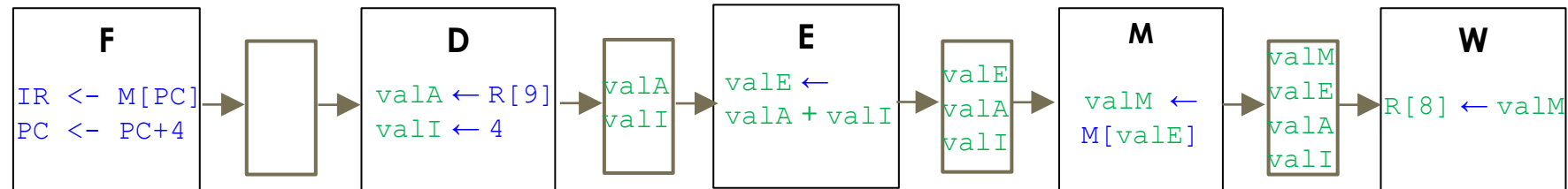
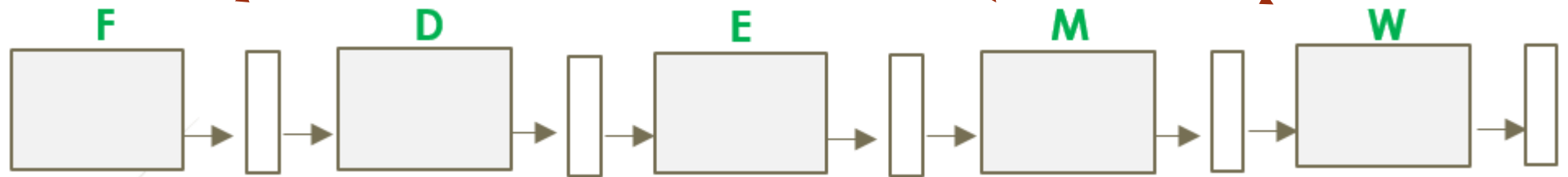
- Start executing a new instruction at every clock cycle
- **Effect:** Different stages of different instructions executed at the same time (at every tick of the clock)

Pipelined Execution – complete picture

Clock cycle:



A closer look:



lw \$t0, 4(\$t1)

add \$t0, \$s1, \$s2

Analysis – Comparing all 3 microprocessor designs (3 models of microprocessor instruction execution)

Sequential Execution – 1 stage:

1. Minimum clock cycle: $400 \text{ ps} + 20 \text{ ps} = 420 \text{ ps}$
2. Latency:
 - $400 \text{ ps} + 20 \text{ ps} = 420 \text{ ps}$
3. CPU throughput: $1 / 420 \text{ ps} = 2.38 \text{ GIPS}$
4. CPI: 1

Staged Execution – 5 stages:

1. Minimum clock cycle: $80 \text{ ps} + 20 \text{ ps} = 100 \text{ ps}$
2. Latency:
 - $(400/5 \text{ ps} + 20 \text{ ps})5 = 500 \text{ ps}$
3. CPU throughput: $1 / 5 \text{ stages} \times 100 \text{ ps} = 2 \text{ GIPS}$
4. CPI: 5

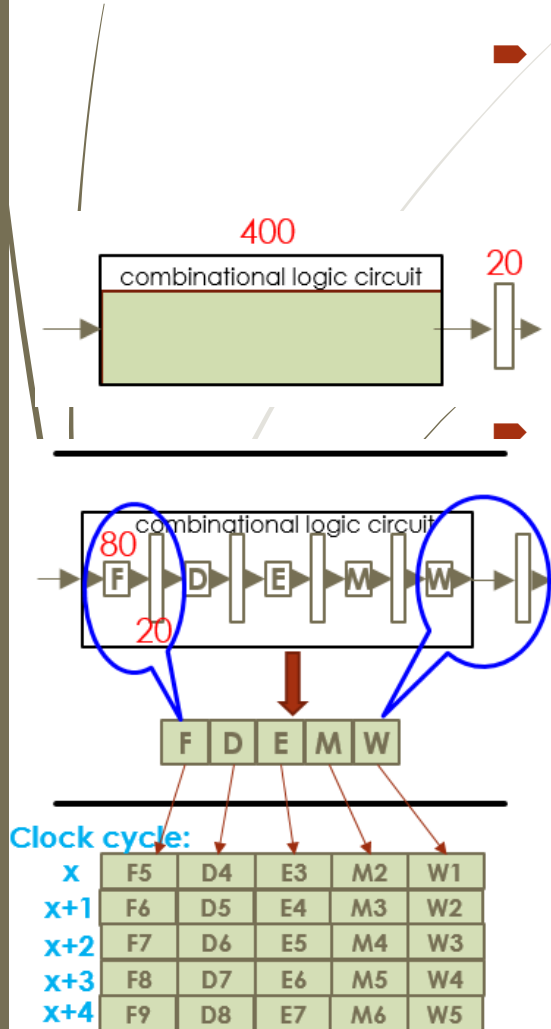
Pipelined Execution – 5 stages:

1. Minimum clock cycle: $80 \text{ ps} + 20 \text{ ps} = 100 \text{ ps}$ <- has not changed from Staged Execution!
2. Latency - cost per instruction: $(400/5 \text{ ps} + 20 \text{ ps})5 = 500 \text{ ps}$ <- has not changed!
3. CPU throughput: $1 / 100 \text{ ps} = 10 \text{ GIPS}$ (or $5 / 500 \text{ ps} = 10 \text{ GIPS}$)
4. CPI: 1 (in steady state)

Conclusion:

Sequential->Staged->Pipelined

- Min clock cycle: ↓
- Instruction latency: ↑
- CPU throughput: ↑
- CPI: ↑ then ↓



Limitations of pipelining - 1

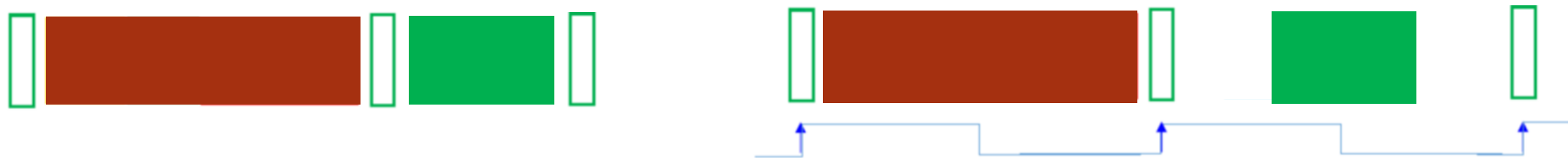
1. Would doubling the number of stages (5 -> 10) double the throughput (10 GIPS -> 20 GIPS)?

- Cost (latency) of each stage
Let's try:
- Minimum clock cycle:
- Microprocessor throughput ->
- So?

Limitations of pipelining - 2

2. What if stages had different propagation delays (latencies)?

- Overall propagation delay (**latency**) of the whole datapath of microprocessor (i.e., sum of propagation delays through all the stages) remains the same
- But some of the stages may have longer propagation delay (latency) than the other stages



- Effect: **clock cycle** would then be limited by propagation delay (latency) of the slowest stage

Limitations of pipelining - 3 - Hazards

3. When next instruction cannot execute in the following clock cycle due to ...
 1. Structural hazard
 2. Data hazard
 3. Control hazard

Hazard 1: Structural Hazard

Pipeline registers not included in these diagrams! Assume there are pipeline registers between each stage and after W stage.

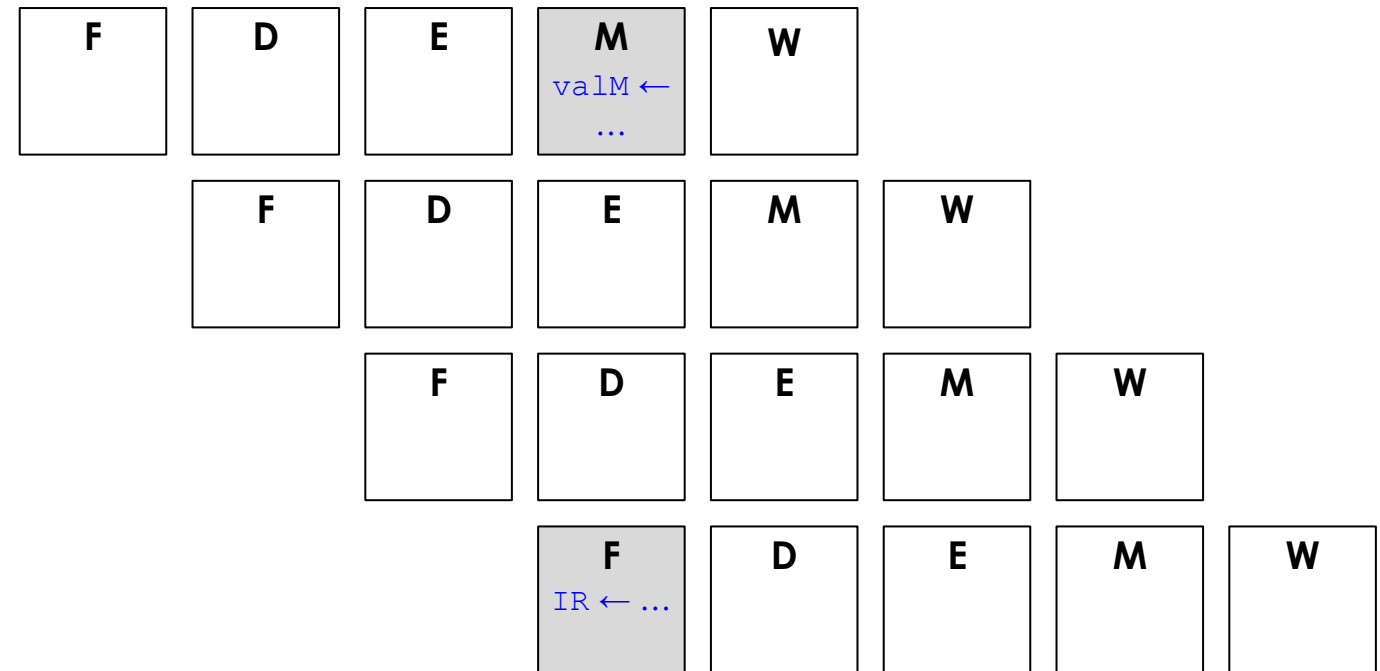
➤ 2 memory accesses (2 read's) done at the same time

`lw $s2, 4($sp)`

`add $s3, $s1, $s2`

`sub $s4, $s1, $s2`

`mul $s5, $s3, $s4`



MIPS deals with this hazard by having 2 memories:

- Instruction memory
- Data memory

MIPS

Datapath with Pipeline Registers

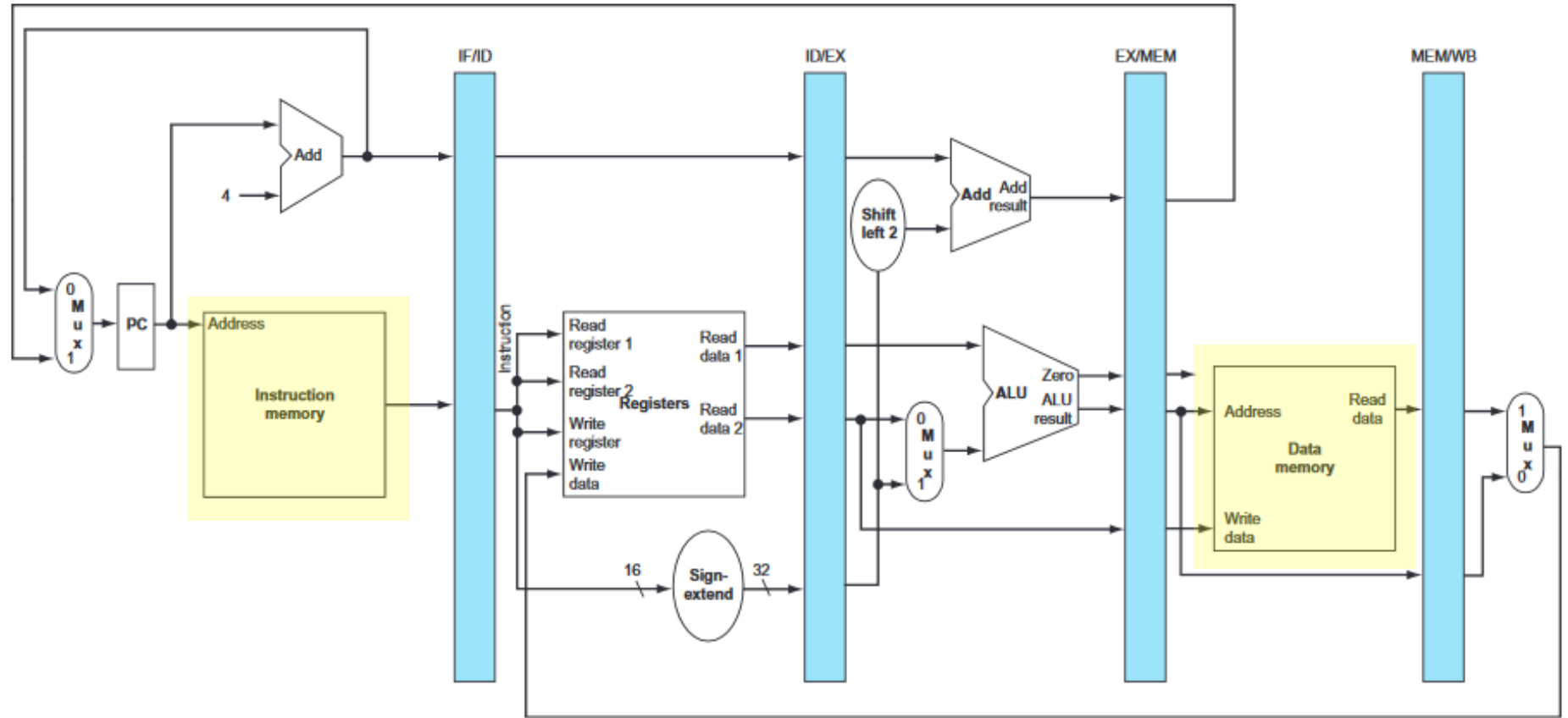
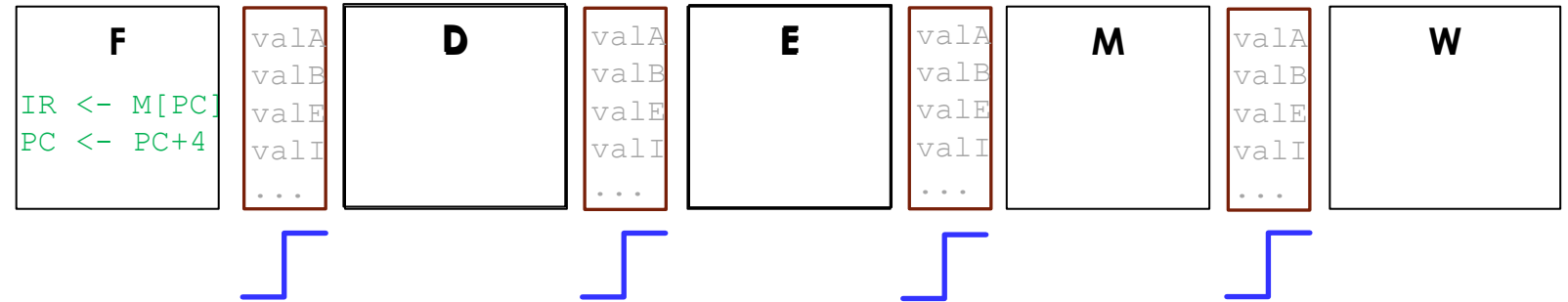


FIGURE 4.35 The pipelined version of the datapath in Figure 4.33. The pipeline registers, in color, separate each pipeline stage. They are labeled by the stages that they separate; for example, the first is labeled IF/ID because it separates the instruction fetch and instruction decode stages. The registers must be wide enough to store all the data corresponding to the lines that go through them.

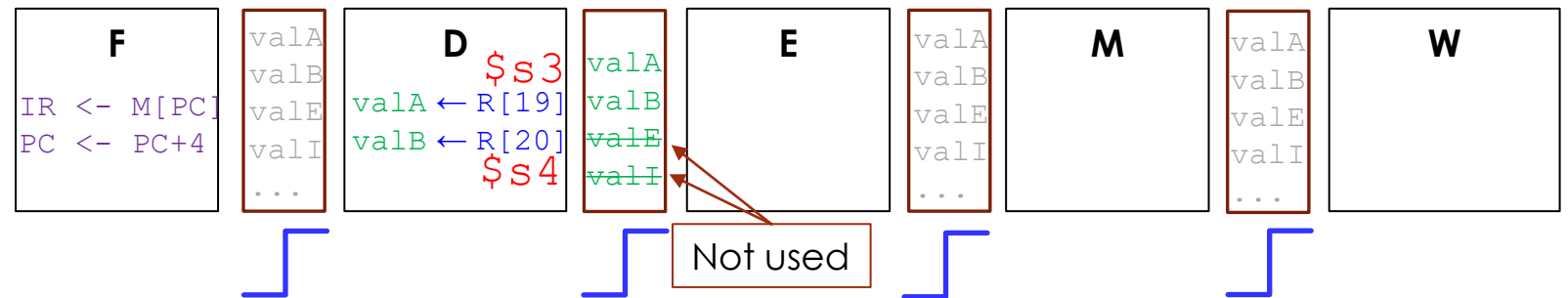
Before we present **Hazard 2**, let's have a close look at these 2 instructions and one way of drawing the pipeline:

Drawing the pipeline

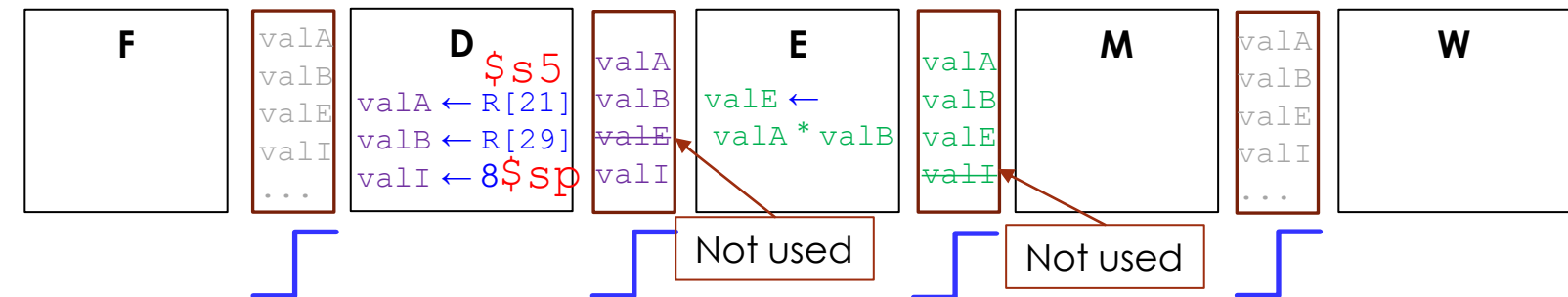
Clock cycle 1



Clock cycle 2



Clock cycle 3



mul \$s5, \$s3, \$s4
sw \$s5, 8(\$sp)

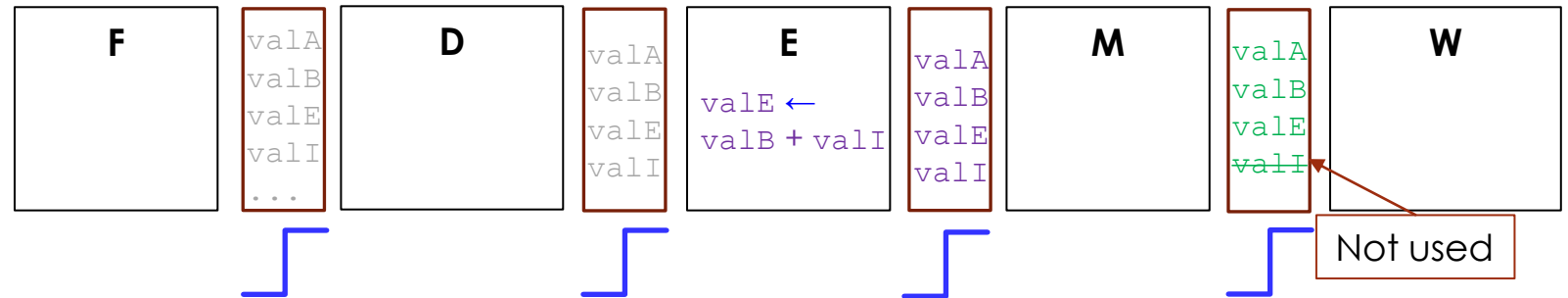
mul \$s5, \$s3, \$s4
sw \$s5, 8(\$sp)

mul \$s5, \$s3, \$s4
sw \$s5, 8(\$sp)

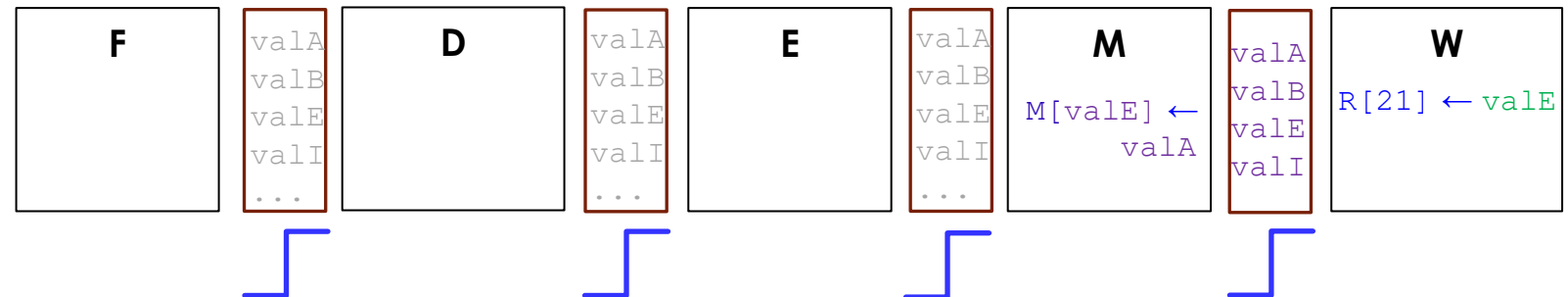
Drawing the pipeline (cont'd)

`mul $s5, $s3, $s4`
`sw $s5, 8($sp)`

Clock cycle 4



Clock cycle 5



Hazard 2: Data Hazard

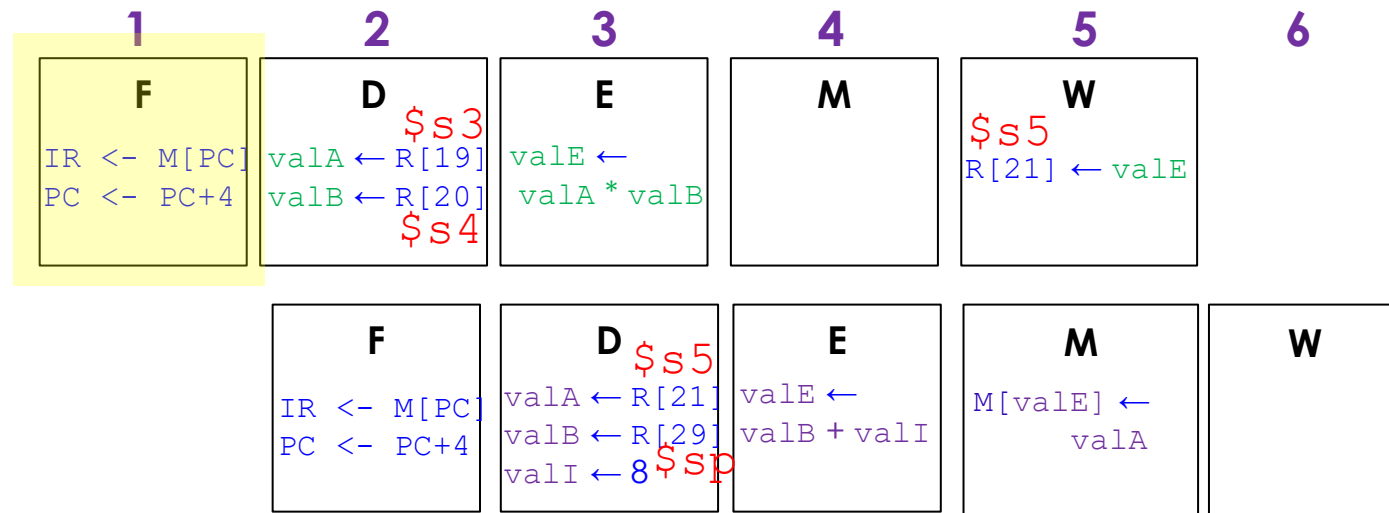
- When stages are dependent on each other for data
->**Data dependency**
- Operand needed by an instruction is currently being computed in the pipeline by another instruction
- Scenario: `mul $s5, $s3, $s4`
`sw $s5, 8($sp)`

Here is another way of drawing the pipeline:

`mul $s5, $s3, $s4`

`sw $s5, 8($sp)`

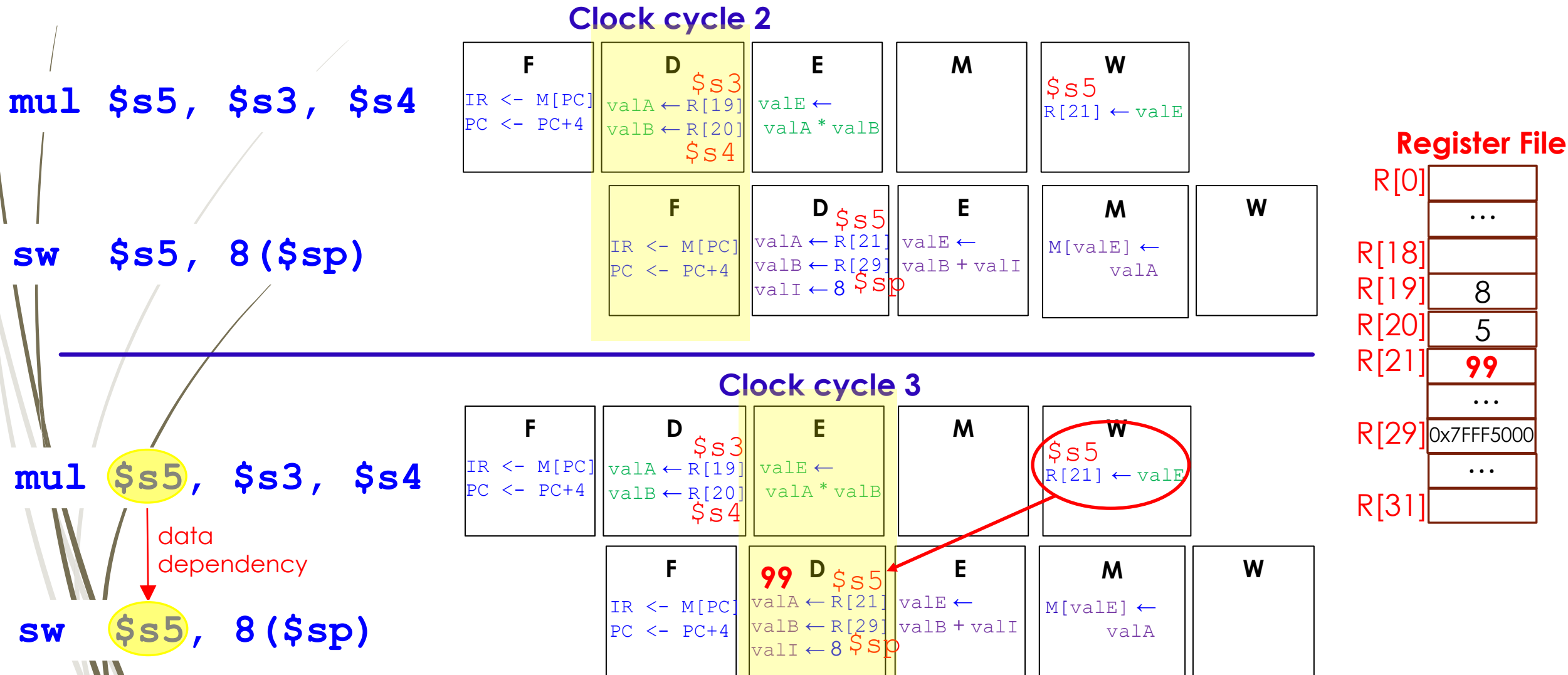
Clock cycle



Register File

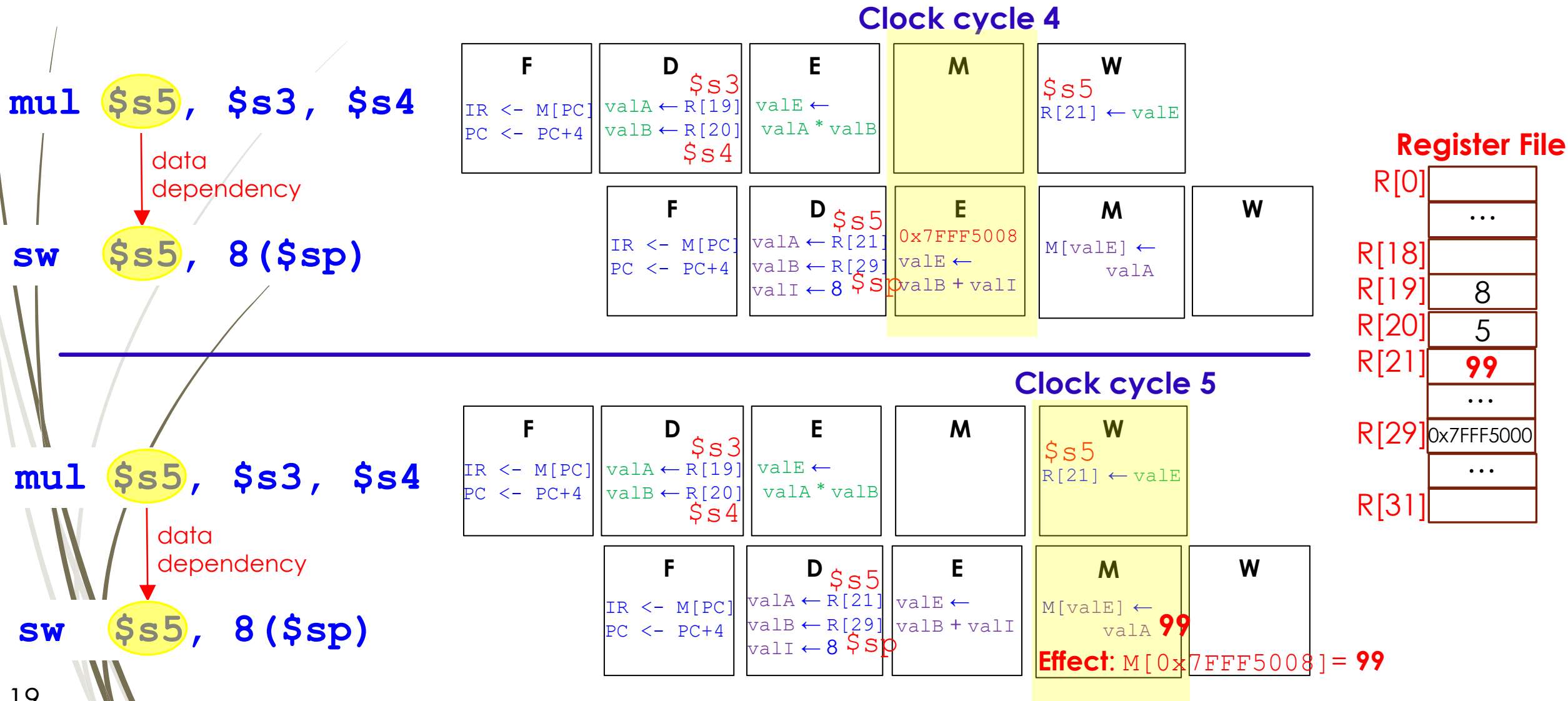
R[0]	
	...
R[18]	
R[19]	8
R[20]	5
R[21]	99
	...
R[29]	0x7FFF5000
	...
R[31]	

Let's see what happens in the pipeline! 1 of 2



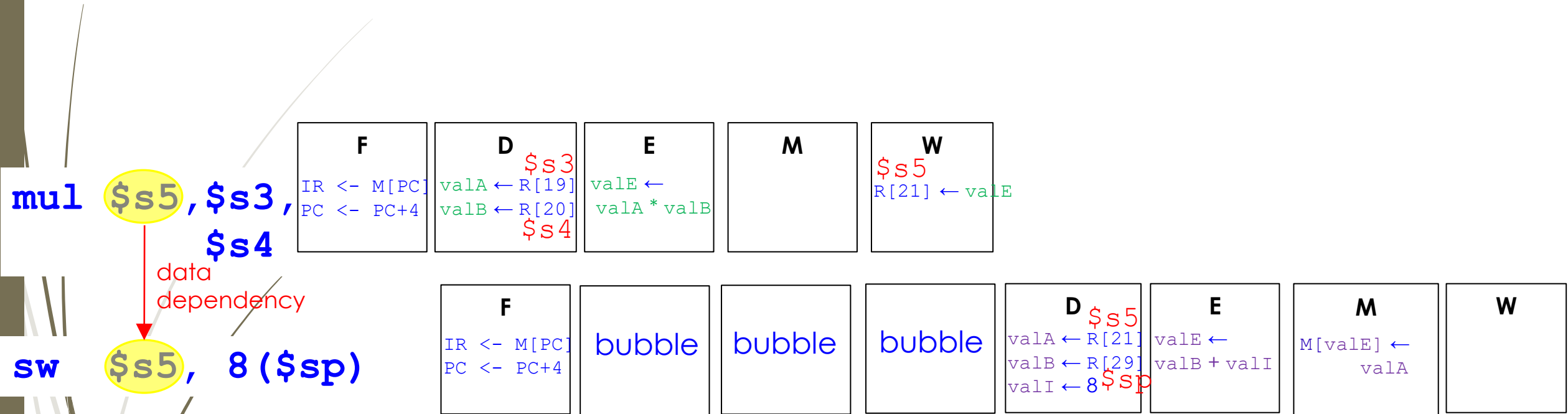
Problem: At Stage D of `sw` (Stage E of `mul`), `sw` needs value of `$s5`. But `mul` stores value it has computed into `$s5` only at its Stage W (Stage M of `sw`), i.e., 2 "ticks" later! => **Too late!!!**

Let's see what happens in the pipeline! 2 of 2



***C or assembly programmer expects **40** to be stored in M[0x7FFF5008] ☹

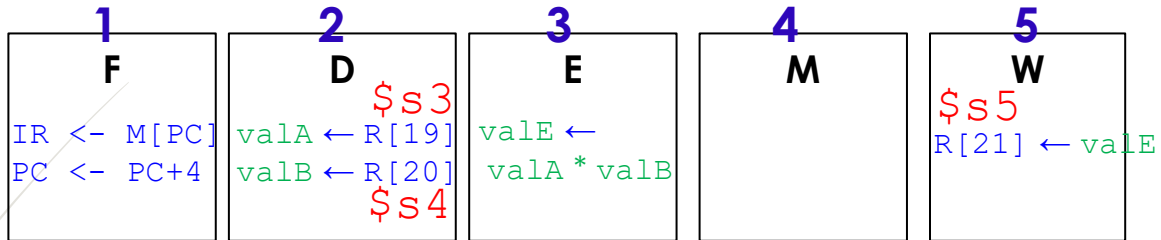
Solution 1: Stall pipeline for instruction `sw`



How *bubbles* are implemented (1 of 3)

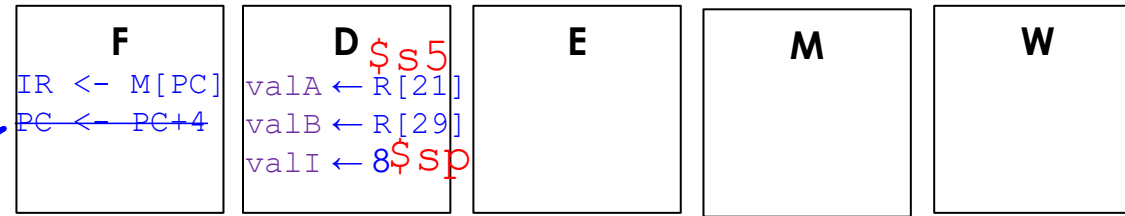
Clock cycle:

**mul \$s5,
\$s3,
\$s4**

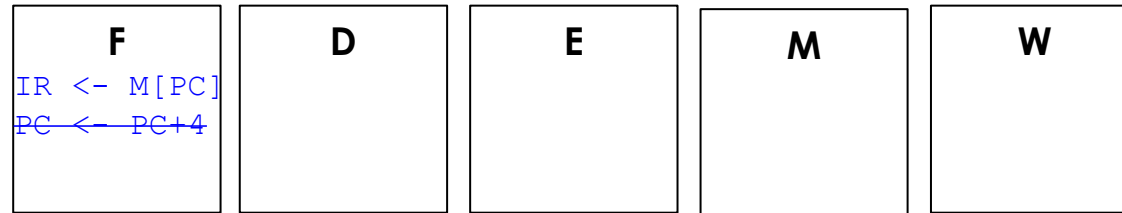


sw \$s5, 8(\$sp)

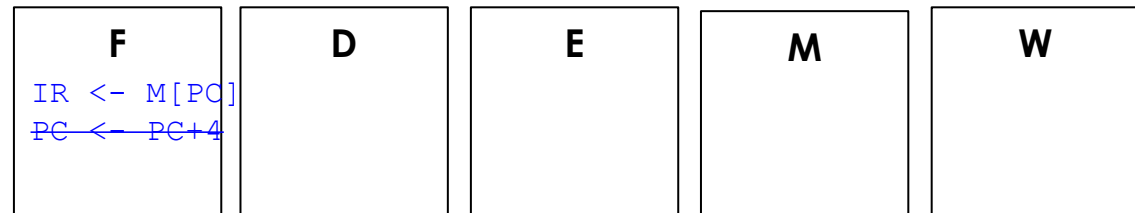
This is how it looks!



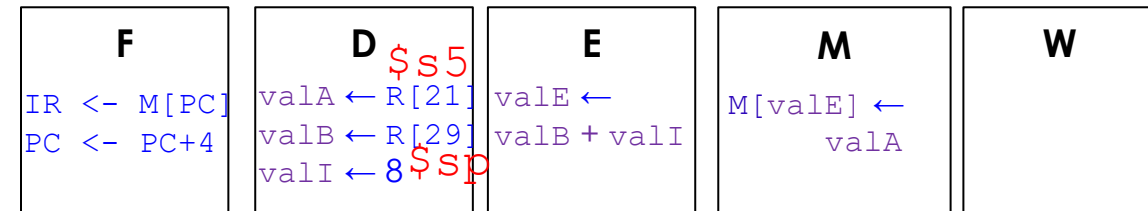
sw \$s5, 8(\$sp)



sw \$s5, 8(\$sp)



sw \$s5, 8(\$sp)



How *bubbles* are implemented (2 of 3)

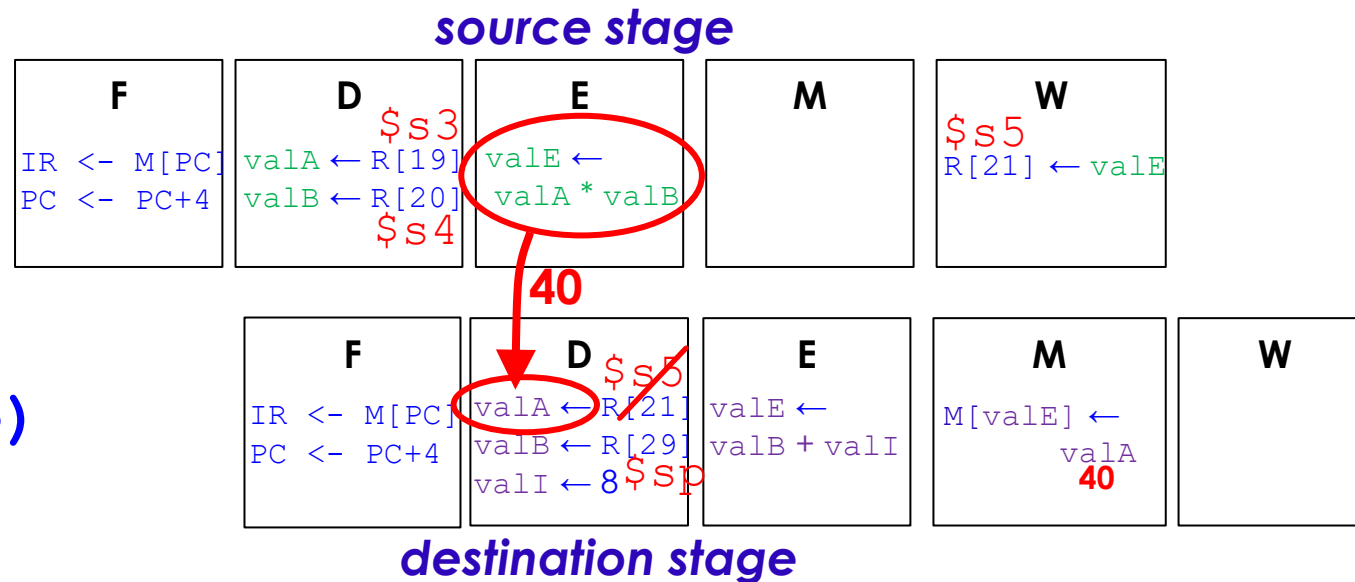
- If instruction `sw` in **D** needs to be stalled (until `mul` executes it **W** stage) then all subsequent instructions after `sw` also need to be stalled
 - Otherwise, we would lose all the next fetched instructions – because `PC` is incremented at each clock cycle (clock cycle does not stall)
- To prevent the instruction `sw` along with the subsequent instructions from making progress through the pipeline
 - 1) The microprocessor prevents the `PC` from being updated
 - Therefore, at the next clock cycle, in **F**, the same instruction `sw` is read once again using the unchanged `PC` value
 - 2) The microprocessor also disables the **load** (or **enable**) lines (set to 0) of the pipeline registers (memory components) in **E**, **M**, and **W** stages
 - This way, all remembered values of pipelines registers are kept unchanged
- The microprocessor does 1) and 2) until `mul` executes it **W** stage then the microprocessor updates the `PC` and proceeds on decoding and executing the next instruction, i.e., the instruction already in IR: `sw $s5, 8($sp)`

Solution 2: Data forwarding (bypassing)

- Forward the data calculated in the *pipeline*, i.e., in one of the stages of the datapath of the microprocessor

`mul $s5, $s3, $s4`
`sw $s5, 8($sp)`

data dependency



Register File

R[0]	
...	
R[18]	
R[19]	8
R[20]	5
R[21]	40
...	
R[29]	0x7FFF5000
...	
R[31]	

Data can only be forwarded from one stage to another where the *destination stage* is either executing at the same time (i.e., at the same clock cycle) as the *source stage* or later **but not before.**

Solution 2: Data forwarding (bypassing)

Adding the
pipeline
registers:

mul **\$s5**, **\$s3**, **\$s4**

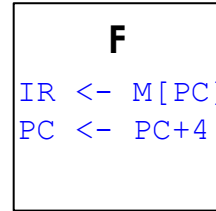
data
dependency

sw

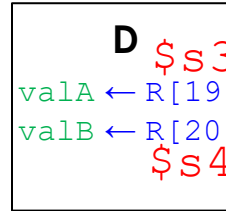
\$s5, **8 (\$sp)**

Clock cycle

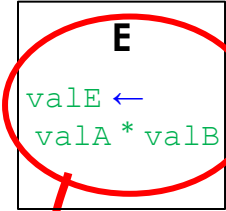
1



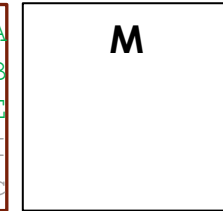
2



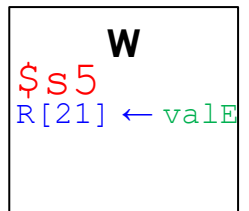
3



4

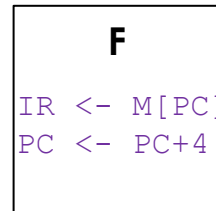


5

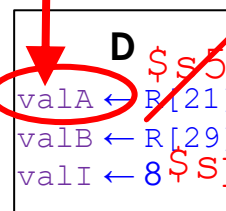


source stage

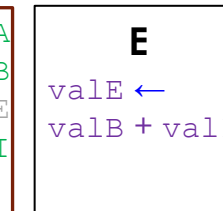
F



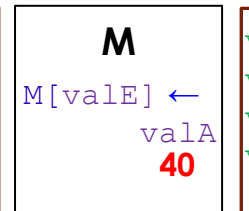
3



E



M



destination stage

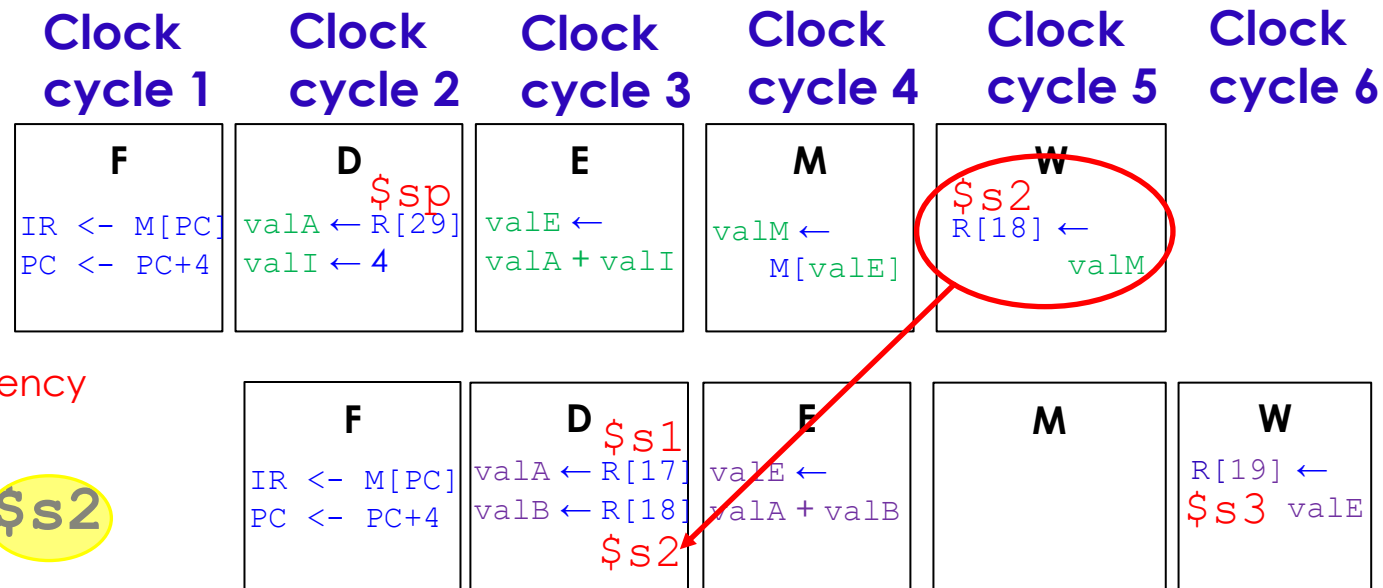
Specific type of data hazard: **Load-to-use**

- **Load-to-use** -> specific combination of instructions (e.g., **lw** then **add**)
- Operand is **used** before it is **loaded** from memory
- Can this type of data hazard be solved using data forwarding? **Solution 2**

lw **\$s2**, 4(\$sp)

add \$s3, \$s1, **\$s2**

data
dependency



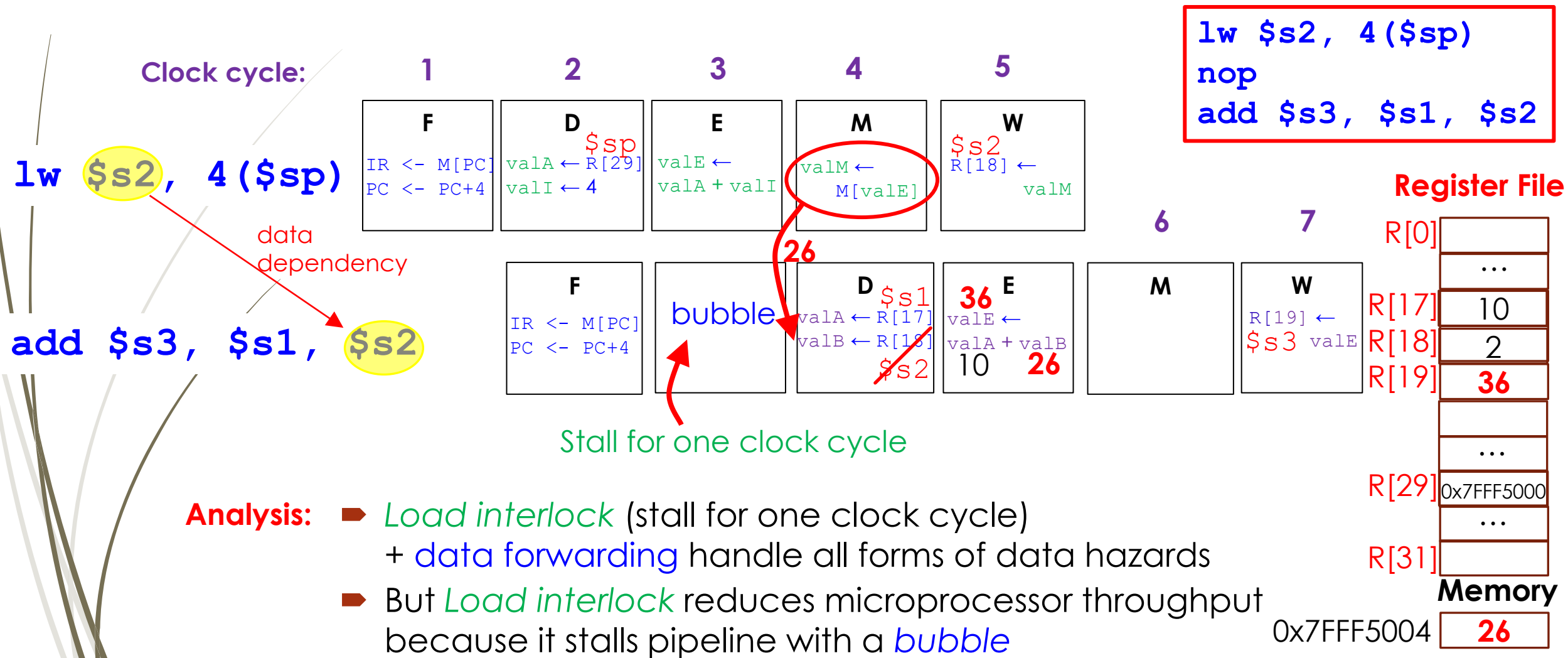
Register File

R[0]	
...	
R[17]	10
R[18]	2
R[19]	12
...	
R[29]	0x7FFF5000
...	
R[31]	

Memory

0x7FFF5004 **26**

Solution 1: *Load interlock*



- Analysis:**
- *Load interlock* (stall for one clock cycle) + *data forwarding* handle all forms of data hazards
 - But *Load interlock* reduces microprocessor throughput because it stalls pipeline with a *bubble*
 - Conclusion: *Load interlock* + *data forwarding* can **nearly** achieve the throughput goal of executing one new instruction at every clock cycle

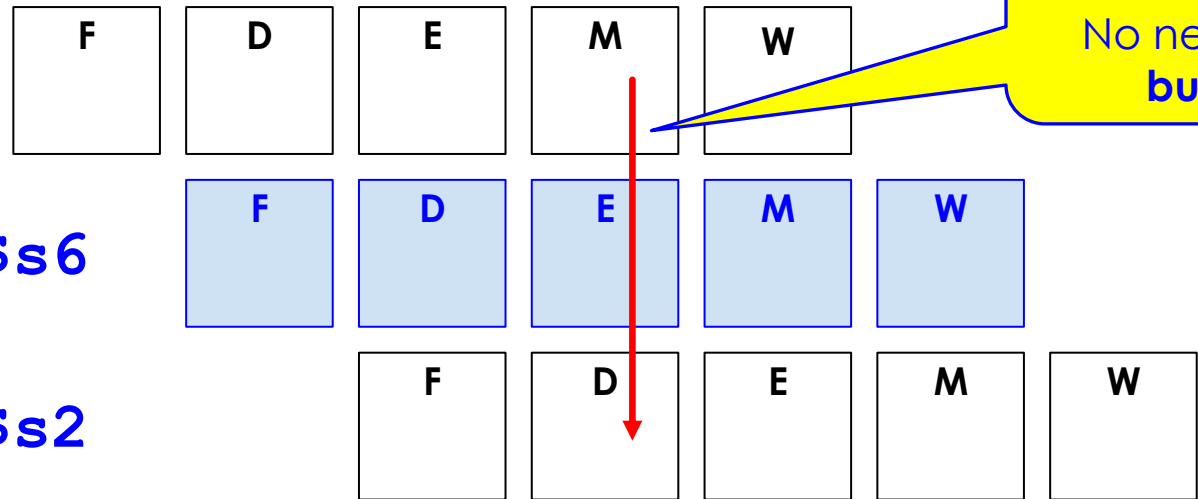
Solution 2: Interleaving

- Instructions are reordered: interleaved with independent instruction

`lw $s2, 4($sp)`

`mul $s7, $s5, $s6`

`add $s3, $s1, $s2`



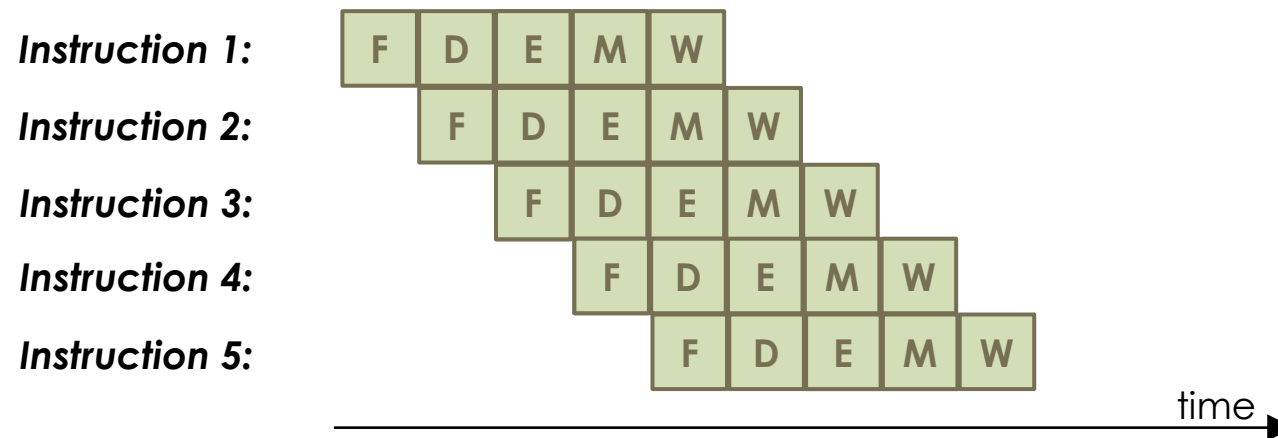
Only need
Data Forwarding!
No need for a
bubble!

Summary - 1

Microprocessor machine code instruction execution:

Model 3. Pipelined execution of machine code instructions

- Start executing 1 instruction at each clock cycle
- Effect: overlap different stages as different instructions are executing



Steady state:

With **n** stages,
now executing
n instructions in
n clock cycles

- Analysis of **pipelined execution** using best case pipeline scenario, i.e., all stages have the same propagation delay, and “steady-state”
 - **Result:** Higher **throughput**

Summary - 2

Microprocessor machine code instruction execution:

3. Pipelined execution of machine code instructions

➤ Limitations on pipelined execution

1. Doubling the number of stages does not double the **throughput**
2. Non-uniform propagation delay of stages: all stages may not have the same propagation delay – stage with longest propagation delay (slowest stage) limits **clock cycle**
3. Hazard: 1) **Structural**, 2) **Data** and 3) **Control**
 1. **Structural Hazard**: when 2 stages attempt to use same hardware resource at same time, e.g., memory
 - **Solution**: **MIPS** has *Data Memory* and *Instruction Memory*
 2. **Data Hazard** created by data dependencies
 - Solution 1: Stall (bubbles)
 - Solution 2: Data forwarding
 - Specific type of Data Hazard: **Load-to-use**
 - Solution 1: Load interlock
 - Solution 2: Interleaving (reordering) instructions

Next Lecture

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design principles
 - Look at an example of an instruction set: MIPS
 - Create our own
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine code instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine code instructions
 - Staged execution of machine code instructions
 - Pipelined execution of machine code instructions + Hazards (cont'd)